

Coding SOP for Interns

School Transport Operations System (TOS) MVP — Backend Implementation Standards & Architecture (XML)

Date: February 18, 2026

Scope: Backend coding SOP (folder structure, naming standards, PR checklist, test checklist) and an XML reference for backend architecture aligned to the TOS MVP PRD.

1. Repository and Folder Structure (Backend)

Recommended approach for interns: Modular Monolith (keeps complexity low while maintaining clean boundaries).

Rules:

- Interns work primarily inside `src/modules/<module_name>/` unless changes are required in config or shared utilities.
- Each module follows: controller → service → repo → model (business rules belong in service).
- Avoid cross-module imports except through `src/common/` abstractions.

Reference folder tree:

```
backend/  
  README.md  
  .env.example  
  package.json / pyproject.toml  
  src/  
    main.ts / main.py  
    config/  
      env.ts  
      logger.ts  
      db.ts  
      redis.ts  
      rbac.ts  
    common/  
      constants/  
      errors/  
      middleware/  
      utils/  
      types/  
    modules/  
      auth/  
        auth.controller.ts  
        auth.service.ts  
        auth.routes.ts  
        auth.dto.ts  
        auth.middleware.ts  
      users/
```

```
routes/
  routes.controller.ts
  routes.service.ts
  routes.routes.ts
  routes.repo.ts
  routes.model.ts
  routes.dto.ts
trips/
  trips.controller.ts
  trips.service.ts
  trips.routes.ts
  trips.repo.ts
  trips.model.ts
  trips.validators.ts
gps/
  gps.controller.ts
  gps.service.ts
  gps.routes.ts
  gps.repo.ts
  gps.model.ts
attendance/
  attendance.controller.ts
  attendance.service.ts
  attendance.routes.ts
  attendance.repo.ts
  attendance.model.ts
  attendance.audit.ts
notifications/
  notifications.controller.ts
  notifications.service.ts
  notifications.routes.ts
providers/
  whatsapp.provider.ts
notification.repo.ts
notification.model.ts
idempotency.ts
admin/
  admin.controller.ts
  admin.service.ts
  admin.routes.ts
  admin.dto.ts
erp_integration/
  fees.service.ts
  fees.client.ts
  fees.dto.ts
db/
  migrations/
  seeds/
  schema.sql
jobs/
  notification.retry.job.ts
  cleanup.job.ts
tests/
  unit/
  integration/
  e2e/
docs/
  api/
  architecture/
docker/
  Dockerfile
```

```
docker-compose.yml
scripts/
  lint.sh
  test.sh
  migrate.sh
```

2. Naming Conventions (Strict)

2.1 Files

- Folders: kebab-case
- Files: dot.case (example: trips.service.ts, attendance.audit.ts)

2.2 Code

- Classes: PascalCase (TripService, GpsRepo)
- Functions/variables: camelCase (createTrip(), latestLocation)
- Constants: UPPER_SNAKE_CASE (GPS_PING_INTERVAL_SEC)
- DB tables: snake_case plural (gps_logs, notification_log)
- DB columns: snake_case (created_at, trip_type)

2.3 API Naming

- Use nouns and consistent prefixes: /api/v1
- Examples: /api/v1/trips/start, /api/v1/gps/update, /api/v1/attendance/mark, /api/v1/admin/live-trips

3. Branching and Commit Rules

3.1 Branch naming

```
feat/<module>-<short-desc>
fix/<module>-<short-desc>
chore/<desc>
docs/<desc>
```

Examples:

- feat/trips-start-end
- fix/notifications-idempotency
- feat/attendance-admin-audit

3.2 Commit messages (Conventional Commits)

```
feat(trips): create trip on driver start
fix(notifications): enforce unique trip_start batch
test(gps): add integration test for latest location update
docs(architecture): update module boundaries
```

4. Coding Rules (Non-negotiable)

4.1 Service owns business rules

- Controllers must be thin: validate input → call service → return response.
- Idempotency and uniqueness must be enforced in service plus DB constraint.
- Do not rely on UI for enforcement; APIs must be authoritative.

4.2 Critical PRD invariants (must be enforced server-side)

- Trip is created only on driver START (no pre-created trips).
- Only one ACTIVE trip per route + trip_type at a time.
- WhatsApp Trip Start notification is idempotent (one trip → one batch).
- Attendance has no default state (null until marked).
- Driver cannot edit attendance after marking; admin can edit with audit log.
- Parent tracking requires login (no public tracking link).

5. PR Checklist (Intern PR Gate)

Every PR must pass the following checks before review:

A) Code quality

- Follows module boundaries and folder structure.
- No business logic in controllers.
- No duplicated logic across modules; shared logic goes to src/common/.
- Errors use standard error classes; no raw thrown strings.
- Logging added for critical actions (trip start/end, notification batch, GPS ingestion).

B) PRD rule coverage

- Trip creation rule enforced.
- One active trip per route+type enforced.
- Notification idempotency implemented (and DB uniqueness present).
- Attendance lock implemented; admin audit implemented.
- RBAC checks on every endpoint touched.

C) DB and migrations

- Migration included for schema changes.
- DB constraints included (unique keys, foreign keys).
- Indexes added for hot queries (active trips, latest location, trip logs).

D) Tests

- Unit tests for core rules changed in the PR.
- Integration tests for DB constraints and module interactions.
- E2E test for at least one major flow if endpoints touched.

E) Documentation

- docs/api updated if endpoints changed.
- .env.example updated if new environment variables introduced.

6. Test Checklist (What to test per module)

6.1 Trips

- Start trip creates a trip record and returns trip_id.
- Double start does not create duplicates (returns existing or blocks).
- Second active trip for same route+type is prevented.
- End trip closes trip; GPS updates are rejected for ended trips.

6.2 Notifications

- Trip start triggers exactly one WhatsApp batch.
- Retries do not resend (idempotency).
- Provider failure retry does not duplicate sends.
- Notification log stores status and provider message id when available.

6.3 GPS

- GPS updates require active trip.
- Latest location is overwritten correctly (last-write-wins).
- GPS logs append correctly for analytics.
- Parent/admin live tracking returns latest location and updated timestamp.
- Stale detection logic works (admin monitoring).

6.4 Attendance

- No default state: unmarked remains null.
- Driver can mark present/absent once per student per trip.
- Driver cannot update after first mark (server rejects).
- Admin edit updates record and writes audit with reason.
- Audit includes old/new values, editor, and timestamp.

6.5 Parent access

- Parent must be authenticated to view tracking.
- Parent can only view routes tied to their child(ren).
- API prevents route enumeration or cross-tenant access.

7. Backend Architecture (XML Reference)

This XML is the reference architecture aligned to the TOS MVP PRD (modules, APIs, data model, and critical constraints).

```
<TransportOpsSystem name="TOS-MVP" version="1.0">
  <Context>
    <Integration type="ERP">
      <Fees mode="read-only" source="ERP_API"/>
      <Auth source="ERP_SSO_or_InternalAuth"/>
    </Integration>
  </Context>

  <Actors>
    <Actor id="ADMIN"
      capabilities="route_manage,assignments,monitoring,attendance_edit_with_audit"/>
    <Actor id="DRIVER" capabilities="start_end_trip,gps_streaming,attendance_marking,call_parent,view_past_attendance"/>
    <Actor id="PARENT" capabilities="login_required,view_live_tracking"/>
  </Actors>

  <Architecture style="ModularMonolith">
    <Modules>

      <Module name="Auth">
        <Responsibilities>
          <Item>Login flows (driver OTP, parent ERP login)</Item>
          <Item>RBAC enforcement (ADMIN/DRIVER/PARENT)</Item>
          <Item>Tenant isolation</Item>
        </Responsibilities>
        <APIs>
          <API method="POST" path="/api/v1/auth/driver/login" role="DRIVER"/>
          <API method="POST" path="/api/v1/auth/verify-otp" role="DRIVER"/>
          <API method="GET" path="/api/v1/auth/me" role="ADMIN|DRIVER|PARENT"/>
        </APIs>
      </Module>

      <Module name="Routes">
        <Responsibilities>
          <Item>Create routes</Item>
          <Item>Assign students to route</Item>
          <Item>Assign driver to route</Item>
          <Item>Expose assigned routes to driver</Item>
        </Responsibilities>
        <APIs>
          <API method="POST" path="/api/v1/routes" role="ADMIN"/>
          <API method="POST" path="/api/v1/routes/{routeId}/assign-driver" role="ADMIN"/>
          <API method="POST" path="/api/v1/routes/{routeId}/assign-students" role="ADMIN"/>
          <API method="GET" path="/api/v1/driver/routes" role="DRIVER"/>
        </APIs>
      </Module>

      <Module name="Trips">
        <Responsibilities>
          <Item>Create trip at runtime on START</Item>
          <Item>One active trip per route+type constraint</Item>
          <Item>End trip closes active trip</Item>
        </Responsibilities>
        <APIs>
          <API method="POST" path="/api/v1/trips/start" role="DRIVER"/>
          <API method="POST" path="/api/v1/trips/end" role="DRIVER"/>
        </APIs>
      </Module>
    </Modules>
  </Architecture>
</TransportOpsSystem>
```

```

    <API method="GET" path="/api/v1/trips/active" role="ADMIN|DRIVER"/>
  </APIs>
  <Rules>
    <Rule id="TRIP_RUNTIME_ONLY">No pre-created trips; trip created only when driver clicks
    START</Rule>
    <Rule id="ONE_ACTIVE_TRIP_PER_ROUTE_TYPE">At most one ACTIVE trip per (route_id,
    trip_type) at a time</Rule>
  </Rules>
</Module>

<Module name="Notifications">
  <Responsibilities>
    <Item>Send WhatsApp notification batch on trip start only</Item>
    <Item>Idempotency: one trip -> one notification batch</Item>
    <Item>Log provider delivery status</Item>
  </Responsibilities>
  <APIs>
    <API method="POST" path="/api/v1/notify/trip-start" role="INTERNAL"/>
  </APIs>
  <Rules>
    <Rule id="TRIP_START_ONLY">Trigger only on trip start; no ETA; no repeat
    notifications</Rule>
    <Rule id="IDEMPOTENT_BATCH">Unique (trip_id, notification_type='TRIP_START')
    enforced</Rule>
  </Rules>
  <Providers>
    <Provider name="WhatsApp">
      <Mode>CloudAPI_or_BSP</Mode>
      <MessageTemplate>
        <Field>student_name</Field>
        <Field>trip_type</Field>
        <Field>portal_link_login_required</Field>
      </MessageTemplate>
    </Provider>
  </Providers>
</Module>

<Module name="GPS">
  <Responsibilities>
    <Item>Ingest GPS updates every 10-15 sec during active trip</Item>
    <Item>Store latest location for live tracking</Item>
    <Item>Store historical logs for analytics</Item>
  </Responsibilities>
  <APIs>
    <API method="POST" path="/api/v1/gps/update" role="DRIVER"/>
    <API method="GET" path="/api/v1/tracking/live" role="PARENT|ADMIN"/>
  </APIs>
  <Rules>
    <Rule id="ACTIVE_TRIP_REQUIRED">GPS updates accepted only for active trip</Rule>
  </Rules>
</Module>

<Module name="Attendance">
  <Responsibilities>
    <Item>Attendance per trip (pickup/drop separately)</Item>
    <Item>No default state</Item>
    <Item>Driver cannot edit after marking</Item>
    <Item>Admin edits with audit log</Item>
  </Responsibilities>
  <APIs>
    <API method="POST" path="/api/v1/attendance/mark" role="DRIVER"/>

```

```

    <API method="GET" path="/api/v1/attendance" role="ADMIN|DRIVER"/>
    <API method="PATCH" path="/api/v1/admin/attendance/{attendanceId}" role="ADMIN"/>
  </APIs>
  <Rules>
    <Rule id="NO_DEFAULT_ATTENDANCE">Attendance is null until explicitly marked</Rule>
    <Rule id="DRIVER_LOCK_AFTER_MARK">Driver cannot edit once marked</Rule>
    <Rule id="ADMIN_AUDIT_REQUIRED">Admin edits require audit log with reason</Rule>
  </Rules>
</Module>

<Module name="AdminDashboard">
  <Responsibilities>
    <Item>Live buses view (active trips + last GPS)</Item>
    <Item>Attendance monitoring</Item>
    <Item>Driver activity monitoring</Item>
    <Item>Fee display read-only</Item>
  </Responsibilities>
  <APIs>
    <API method="GET" path="/api/v1/admin/live-trips" role="ADMIN"/>
    <API method="GET" path="/api/v1/admin/trips" role="ADMIN"/>
    <API method="GET" path="/api/v1/admin/attendance" role="ADMIN"/>
    <API method="GET" path="/api/v1/admin/fees" role="ADMIN"/>
  </APIs>
</Module>

<Module name="ERPIntegration">
  <Responsibilities>
    <Item>Fetch transport fees read-only from ERP</Item>
  </Responsibilities>
  <APIs>
    <API method="GET" path="/api/v1/erp/fees" role="INTERNAL"/>
  </APIs>
</Module>

</Modules>
</Architecture>

<DataModel>
  <Tables>

    <Table name="routes">
      <Column name="id" type="uuid" key="pk"/>
      <Column name="tenant_id" type="uuid" indexed="true"/>
      <Column name="name" type="varchar"/>
      <Column name="status" type="varchar"/>
      <Column name="created_at" type="timestamp"/>
    </Table>

    <Table name="route_students">
      <Column name="route_id" type="uuid" key="pk_part"/>
      <Column name="student_id" type="uuid" key="pk_part"/>
    </Table>

    <Table name="route_driver_assignment">
      <Column name="route_id" type="uuid" indexed="true"/>
      <Column name="driver_id" type="uuid" indexed="true"/>
      <Column name="active_from" type="timestamp"/>
      <Column name="active_to" type="timestamp" nullable="true"/>
    </Table>

    <Table name="trips">

```

```

    <Column name="id" type="uuid" key="pk"/>
    <Column name="tenant_id" type="uuid" indexed="true"/>
    <Column name="route_id" type="uuid" indexed="true"/>
    <Column name="driver_id" type="uuid" indexed="true"/>
    <Column name="trip_type" type="enum(PICKUP,DROP)" indexed="true"/>
    <Column name="status" type="enum(ACTIVE,ENDED)" indexed="true"/>
    <Column name="start_time" type="timestamp"/>
    <Column name="end_time" type="timestamp" nullable="true"/>
</Table>

<Table name="attendance">
    <Column name="id" type="uuid" key="pk"/>
    <Column name="trip_id" type="uuid" indexed="true"/>
    <Column name="student_id" type="uuid" indexed="true"/>
    <Column name="status" type="enum(PRESENT,ABSENT)" nullable="true"/>
    <Column name="marked_by" type="uuid"/>
    <Column name="marked_at" type="timestamp"/>
    <Column name="locked" type="boolean" default="true"/>
</Table>

<Table name="attendance_audit">
    <Column name="id" type="uuid" key="pk"/>
    <Column name="attendance_id" type="uuid" indexed="true"/>
    <Column name="edited_by" type="uuid"/>
    <Column name="old_status" type="enum(PRESENT,ABSENT)" nullable="true"/>
    <Column name="new_status" type="enum(PRESENT,ABSENT)" nullable="true"/>
    <Column name="reason" type="varchar"/>
    <Column name="edited_at" type="timestamp"/>
</Table>

<Table name="latest_bus_location">
    <Column name="trip_id" type="uuid" key="pk"/>
    <Column name="route_id" type="uuid" indexed="true"/>
    <Column name="driver_id" type="uuid"/>
    <Column name="lat" type="decimal"/>
    <Column name="lng" type="decimal"/>
    <Column name="accuracy_m" type="decimal" nullable="true"/>
    <Column name="updated_at" type="timestamp" indexed="true"/>
</Table>

<Table name="gps_logs">
    <Column name="id" type="uuid" key="pk"/>
    <Column name="trip_id" type="uuid" indexed="true"/>
    <Column name="timestamp" type="timestamp"/>
    <Column name="lat" type="decimal"/>
    <Column name="lng" type="decimal"/>
    <Column name="accuracy_m" type="decimal" nullable="true"/>
    <Column name="received_at" type="timestamp" indexed="true"/>
</Table>

<Table name="notification_log">
    <Column name="id" type="uuid" key="pk"/>
    <Column name="trip_id" type="uuid" indexed="true"/>
    <Column name="type" type="enum(TRIP_START)" indexed="true"/>
    <Column name="status" type="enum(QUEUED,SENT,FAILED)" indexed="true"/>
    <Column name="provider_message_id" type="varchar" nullable="true"/>
    <Column name="created_at" type="timestamp"/>
</Table>

<Constraints>
    <Constraint name="uniq_trip_start_notification" table="notification_log">

```

```
    <Unique columns="trip_id,type"/>
  </Constraint>

  <Constraint name="one_active_trip_per_route_type" table="trips">
    <Note>Enforce via partial unique index: (route_id, trip_type) WHERE status='ACTIVE'</Note>
  </Constraint>
</Constraints>

</Tables>
</DataModel>

</TransportOpsSystem>
```